	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Projektowanie serwisów sieciowych		
Prowadzący	mgr inż. Leonid Rusanov		
Temat	Autentykacja i Middleware		
Student (+ nr indeksu)	Dawid Wendt 122652		
Nr lab.	7	Data oddania spr.	07.01.2026

1. Cel ćwiczenia

Celem ćwiczenia jest zapoznanie się z systemem autentykacji w Laravelu, tworzeniem niestandardowych middleware'ów, kontrolą dostępu do tras oraz implementacją ról użytkowników.

2. Przebieg

W pierwszej kolejności zainstalowało się Laravel zapoczątkowało projekt i przetestowało działanie wstępne strony

```

INFO: Preparing database.

Creating migration table ..... 9.98ms DONE

INFO: Running migrations.

0001_01_01_000000_create_users_table ..... 20.81ms DONE
0001_01_01_000001_create_cache_table ..... 7.67ms DONE
0001_01_01_000002_create_jobs_table ..... 16.74ms DONE

PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7> cd lab07-app
PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app>

```

```

PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> composer require laravel/breeze --dev
./composer.json has been updated
Running composer update laravel/breeze
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking laravel/breeze (v2.3.8)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Downloading laravel/breeze (v2.3.8)
  - Installing laravel/breeze (v2.3.8): Extracting archive
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

 INFO  Discovering packages.

laravel/breeze ..... DONE
laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

81 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

 INFO  No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version ^2.3 for laravel/breeze
PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app>

```

```

Using version ^2.3 for laravel/breeze
PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> php artisan breeze:install

Which Breeze stack would you like to install?
Blade with Alpine ..... blade
Livewire (Volt Class API) with Alpine ..... livewire
Livewire (Volt Functional API) with Alpine ..... livewire-functional
React with Inertia ..... react
Vue with Inertia ..... vue
API only ..... api
) blade

Would you like dark mode support? (yes/no) [no]
)

Which testing framework do you prefer? [Pest]
Pest ..... 0
PHPUnit ..... 1
)

./composer.json has been updated
Running composer update phpunit/phpunit
Loading composer repositories with package information
Updating dependencies
Lock file operations: 0 installs, 0 updates, 26 removals

```

VITE v7.3.1 ready in 396 ms

- Local: <http://localhost:5173/>
- Network: use `--host` to expose
- press `h + enter` to show help

LARAVEL v12.45.1 plugin v2.0.1

- APP_URL: <http://localhost>

php artisan migrate

PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> **php** artisan migrate

INFO Nothing to migrate.

PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> **php** artisan serve

INFO Server running on [<http://127.0.0.1:8000>].

Press Ctrl+C to stop the server

```
2026-01-07 14:17:07 / .....
2026-01-07 14:17:07 /build/assets/app-CEwZte8_.css .....
2026-01-07 14:17:07 /build/assets/app-CiZ6hk-B.js .....
2026-01-07 14:17:08 /favicon.ico .....
```



Name

Email

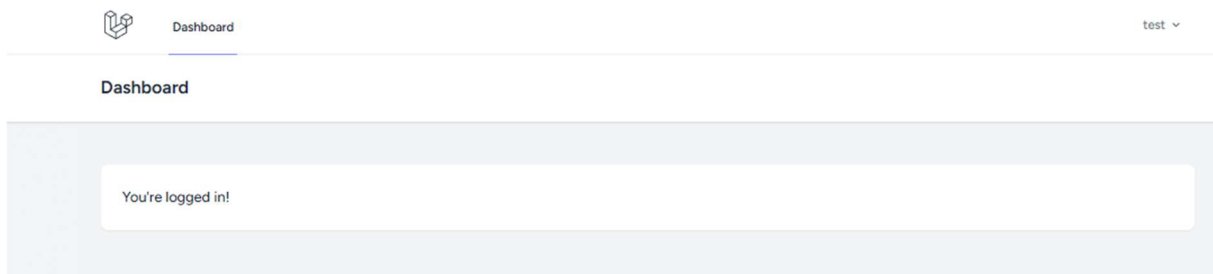
Password

The password field must be at least 8 characters.

Confirm Password

[Already registered?](#) **REGISTER**

Reregistracja wymagała hasła 8 znakowego co najmniej.



Rejestracja i logowanie zadziałały poprawnie

W kolejnej części zadania dodaliśmy rolę admin do bazy danych w pierwszej kolejności musieliśmy dodać pole is_admin do bazy

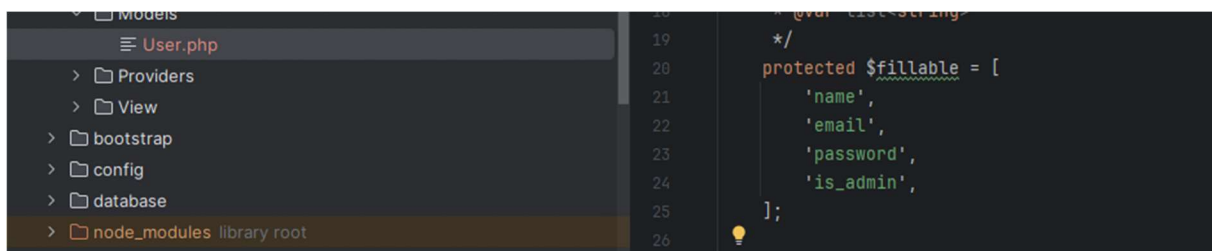
```
PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> php artisan make:migration add_is_admin_to_users_table
[Info] Migration [C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app\database\migrations\2026_01_07_131927_add_is_admin_to_users_table.php] created successfully.
```

```

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->boolean('is_admin')->default(false)->after('email_verified_at');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('is_admin');
        });
    }
};

```



W następnej kolejności stworzyliśmy niestandardowe middleware dzięki któremu będziemy sprawdzać czy aktualny użytkownik jest adminem

```

PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> php artisan make:middleware VerifyAdmin

INFO: Middleware [C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app\app\Http\Middleware\VerifyAdmin.php] created successfully.

```

```

8
9 class VerifyAdmin
10 {
11     public function handle(Request $request, Closure $next): Response
12     {
13         // Sprawdź, czy użytkownik jest zalogowany i czy ma rolę admin
14         if (auth()->check() && auth()->user()->is_admin) {
15             return $next($request);
16         }
17
18         // Jeśli nie - przekieruj na dashboard z błędem
19         return redirect()
20             ->route('dashboard')
21             ->withErrors('Brak uprawnień administratora.');
```

```

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'../../routes/web.php',
        commands: __DIR__.'../../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->alias([
            'admin' => \App\Http\Middleware\VerifyAdmin::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();

```

Kolejno stworzyliśmy kontroler admina

```
PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> php artisan make:controller AdminController
```

```
INFO Controller [C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app\app\Http\Controllers\AdminController.php] created successfully.
```

lab07-app

app

Http

Controllers

Auth

AdminController.php

Controller.php

ProfileController.php

Middleware

VerifyAdmin.php

Requests

Models

Providers

View

bootstrap

config

database

```

35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53

```

```

* Przetacz status admina dla użytkownika
*/
public function toggleAdmin(User $user): RedirectResponse
{
    if (auth()->id() === $user->id) {
        return redirect()
            ->route('admin.users')
            ->withErrors('Nie możesz odebrać sobie uprawnień administratora.');
```

```

    }

    $user->update(['is_admin' => !$user->is_admin]);

    $status = $user->is_admin ? 'nadane' : 'usunięte';

    return redirect()->route('admin.users')
        ->with('success',
            'Uprawnienia dla użytkownika ' . $user->name . ' zostały ' . $status . '.');
```

Oraz zdefiniowanie tras dla naszego panelu

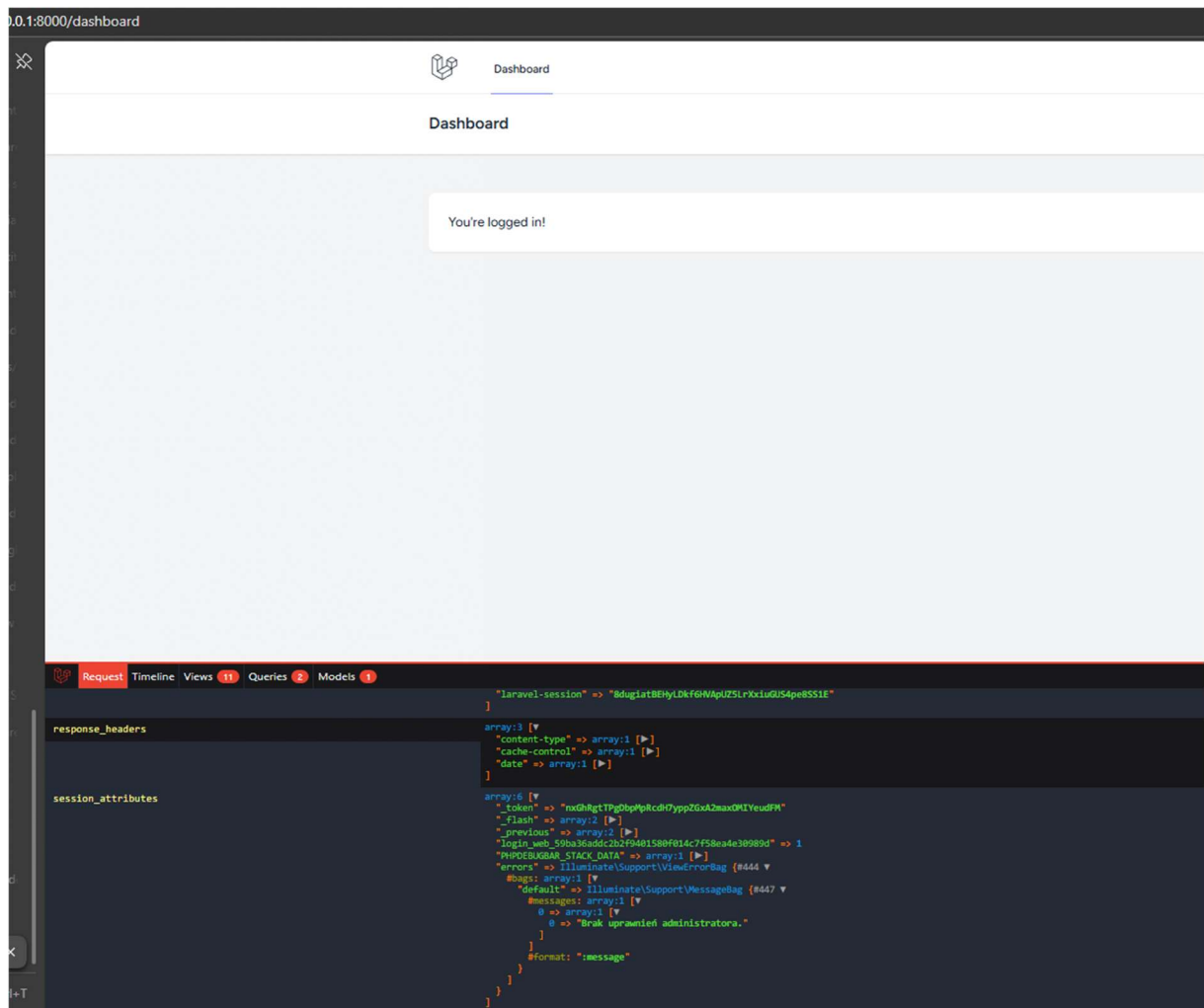
```
Controller.php
ProfileController.php
Middleware
Requests
Models
Providers
View
bootstrap
config
database
node_modules library root
public
resources
routes
auth.php
console.php
web.php
storage
tests
vendor
.editorconfig
.env
.env.example
.gitattributes
.gitignore
artisan
composer.json
composer.lock

10 // Dashboard - dostępny dla zalogowanych użytkowników
11 Route::middleware('auth')->group(function () {
12     Route::get('/dashboard', function () {
13         return view('dashboard');
14     }->name('dashboard'));
15
16     Route::get('/profile', [ProfileController::class, 'edit'])
17         ->name('profile.edit');
18     Route::patch('/profile', [ProfileController::class, 'update'])
19         ->name('profile.update');
20     Route::delete('/profile', [ProfileController::class, 'destroy'])
21         ->name('profile.destroy');
22 });
23
24 // Panel admina - dostępny TYLKO dla administratorów
25 Route::middleware(['auth', 'admin'])->group(function () {
26     Route::get('/admin', [AdminController::class, 'index'])
27         ->name('admin.index');
28     Route::get('/admin/users', [AdminController::class, 'listUsers'])
29         ->name('admin.users');
30     Route::post('/admin/users/{user}/toggle-admin',
31         [AdminController::class, 'toggleAdmin'])
32         ->name('admin.toggle-admin');
33 });
34
35 require __DIR__ . '/auth.php';
36
37
```

Oraz stworzenie widoków dla naszego panelu:

```
css
js
views
admin
index.blade.php
users.blade.php
auth
components

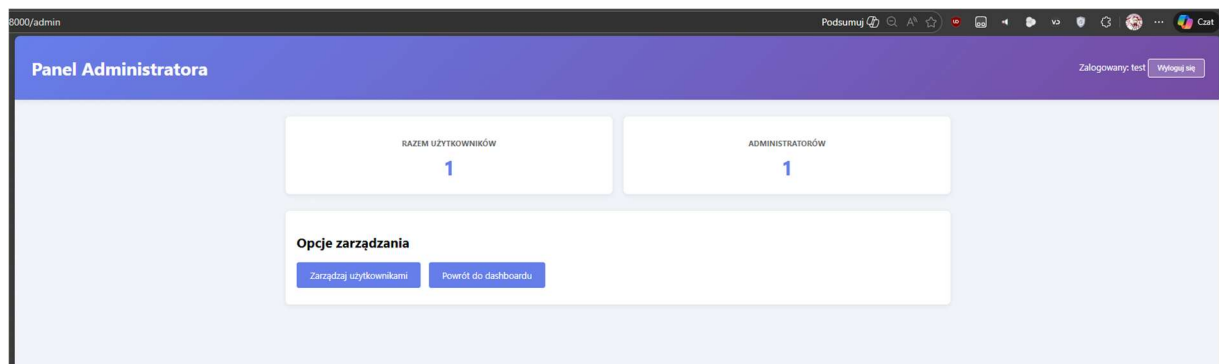
94 <div class="stat-card">
95     <h3>Administratorów</h3>
96     <div class="number">{{ $total_admins }}</div>
97 </div>
98
99
100 <div class="menu">
101     <h2>Opcje zarządzania</h2>
102     <a href="{{ route('admin.users') }}">Zarządzaj użytkownikami</a>
```



Jak widać nie admin nie mógł się zalogować dostaliśmy na dole nasz błąd „Brak uprawnień administratora”

```
> User::first()->update(['is_admin' => true])
[!] Aliasing 'User' to 'App\Models\User' for this Tinker session.
= true
```

Następnie utworzyło się naszego jednego użytkownika adminem



Teraz jak widać nasz użytkownik dostał dostęp do panelu admin

[← Powrót do panelu](#)

✓ Uprawnienia dla użytkownika test2 zostały usunięte.

ID	Imię i nazwisko	Email	Status	Akcje
1	test	dawwen001@pbs.edu.pl	Administrator	Usuń uprawnienia
2	test2	test2@ex.com	Użytkownik	Nadaj uprawnienia

Usuwanie i dodawanie uprawnień działa poprawnie oraz nie ma możliwości usunięcia sobie uprawnień

W ostatniej części ćwiczenia należało utworzyć dodatkowy middleware CheckEmailVerified

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps();
});
```

Jako, że istnieje pole email_verified_at możemy go wykorzystać bez dodawania nowych pól

```
PS C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app> php artisan make:middleware CheckEmailVerified

INFO: Middleware [C:\Users\Dawid\PycharmProjects\PSS_Dawid_Wendt\Zad_7\lab07-app\app\Http\Middleware\CheckEmailVerified.php] created successfully.
```

Tworzymy middleware

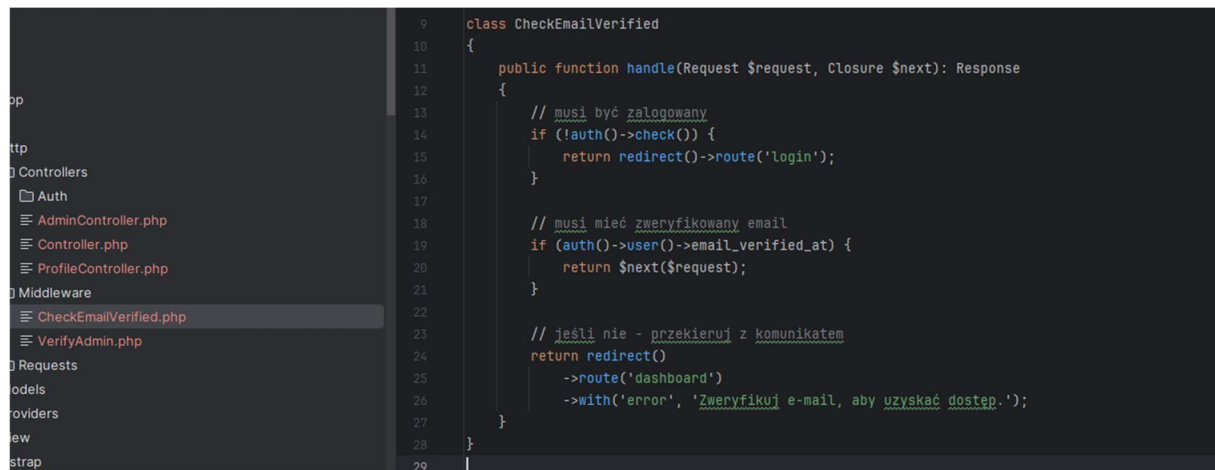
```
app.php
providers.php
config
database
  factories
  migrations
    0001_01_01_000000_create_users_table.php

11 health: '/up',
12 )
13 ->withMiddleware(function (Middleware $middleware) {
14     $middleware->alias([
15         'admin' => \App\Http\Middleware\VerifyAdmin::class,
16         'verified_email' => \App\Http\Middleware\CheckEmailVerified::class,
17     ]);
18 })
```

```
// Dashboard - dostępny dla zalogowanych użytkowników
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', function () {
        return view('dashboard');
    })->name('dashboard');
});

Route::middleware('auth','verified_email')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])
        ->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])
        ->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])
        ->name('profile.destroy');
});
```

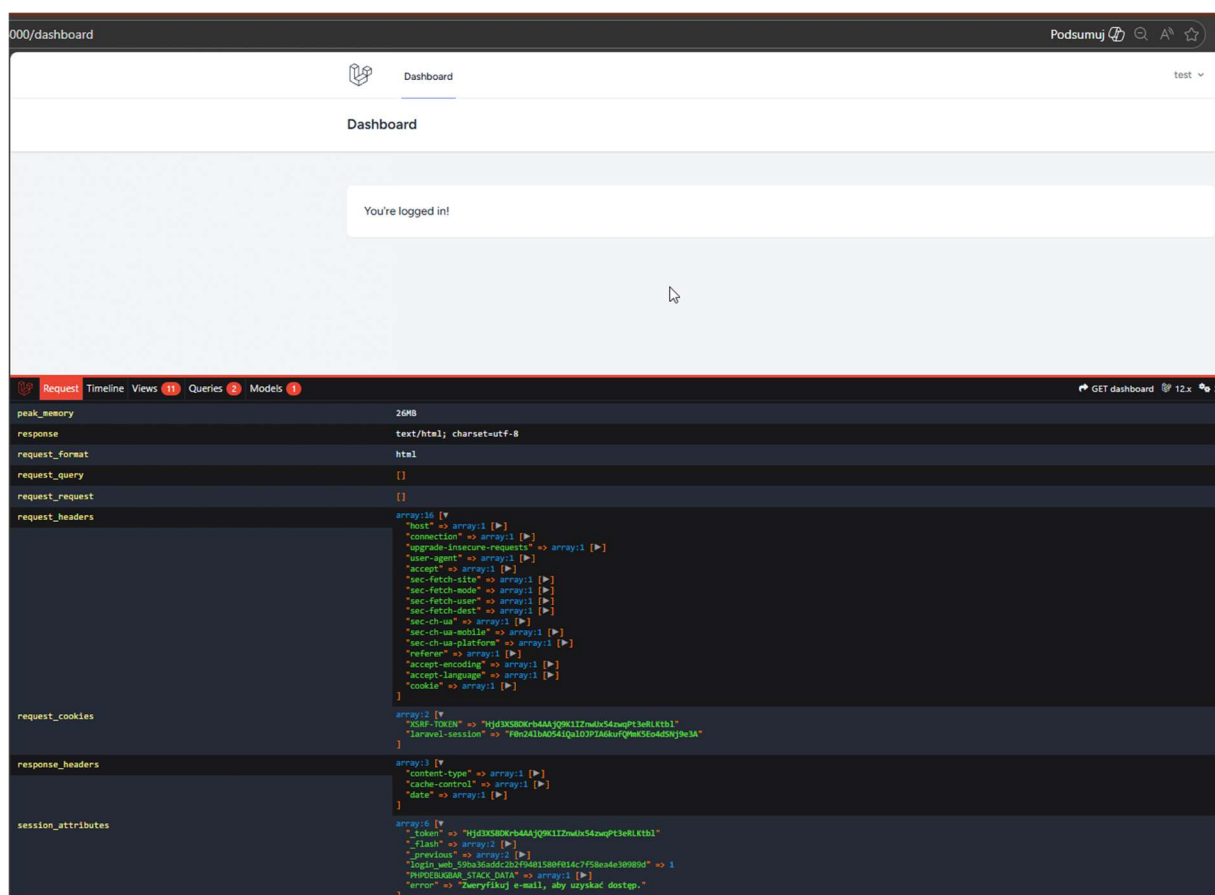
Dodajemy nasz middleware jako wymaganie przykładowo do dostępu do naszego profilu



```
9 class CheckEmailVerified
10 {
11     public function handle(Request $request, Closure $next): Response
12     {
13         // musi być zalogowany
14         if (!auth()->check()) {
15             return redirect()->route('login');
16         }
17
18         // musi mieć zweryfikowany email
19         if (auth()->user()->email_verified_at) {
20             return $next($request);
21         }
22
23         // jeśli nie - przekieruj z komunikatem
24         return redirect()
25             ->route('dashboard')
26             ->with('error', 'Zweryfikuj e-mail, aby uzyskać dostęp.');
```

The screenshot shows an IDE with a file explorer on the left. The file explorer lists the following directories and files: Controllers (Auth, AdminController.php, Controller.php, ProfileController.php), Middleware (CheckEmailVerified.php, VerifyAdmin.php), Requests, Models, Providers, View, and Bootstrap. The main editor displays the code for the CheckEmailVerified middleware class, which implements the handle method to check if the user is authenticated and has a verified email. If not, it redirects to the login page or the dashboard with an error message.

Nasz middleware mamy 2 wymagania bycie zalogowanym jeśli nie to do strony logowania nasz przekieruje w przeciwnym wypadku jeśli nie mamy zweryfikowany email to wroci nas do dashboard z błędem można go w przyszłości wyświetlać odpowiednio komunikat z błędem



Jak widać użytkownik nie może dostać się na swój profil gdyż nie ma zweryfikowanego emailu

3. Wnioski

Podczas wykonywania ćwiczenia udało się zapoznać z mechanizmem autentykacji w frameworku Laravel oraz sposobem kontroli dostępu do zasobów aplikacji przy użyciu middleware. Rejestracja oraz logowanie użytkowników działały poprawnie, a narzucone wymagania dotyczące hasła zwiększają podstawowy poziom bezpieczeństwa aplikacji.

Dodanie pola `is_admin` do bazy danych umożliwiło wprowadzenie prostego systemu ról użytkowników. Dzięki temu możliwe było rozróżnienie zwykłych użytkowników od administratorów oraz ograniczenie dostępu do panelu administracyjnego wyłącznie dla osób posiadających odpowiednie uprawnienia. Utworzenie niestandardowego middleware pozwoliło na czytelne i centralne sprawdzanie tych uprawnień bez potrzeby powielania kodu w kontrolerach.

Zaimplementowany panel administratora działa poprawnie – użytkownik bez uprawnień nie ma do niego dostępu, a komunikat o błędzie jest widoczny po przekierowaniu. Mechanizm nadawania i odbierania uprawnień administratora funkcjonuje zgodnie z założeniami, a dodatkowe zabezpieczenie uniemożliwia odebranie uprawnień samemu sobie, co zapobiega przypadkowemu zablokowaniu dostępu do panelu.

W ostatniej części ćwiczenia utworzono middleware `CheckEmailVerified`, który sprawdza, czy użytkownik ma zweryfikowany adres e-mail. Wykorzystanie istniejącego pola `email_verified_at` pozwoliło na realizację tego zadania bez konieczności modyfikacji struktury bazy danych. Dzięki temu dostęp do wybranych tras został dodatkowo ograniczony, co zwiększa bezpieczeństwo aplikacji.

Ćwiczenie pokazało, że middleware w Laravelu jest skutecznym narzędziem do zarządzania autentykacją, autoryzacją oraz kontrolą dostępu, a ich zastosowanie znacząco poprawia czytelność kodu oraz bezpieczeństwo całej aplikacji.